MPJ lab Assignment - 03

Sayali Patil
1032233901
26 (D1)

PROBLEM STATEMENT :- Write a Java Program to implement method overloading & method overriding in Java.

1. Write a class Shapes. Find the area of basic shapes using polymorphism (constructor overloading & method overloading).
2. Write a Java program to create a parent class Hillstations & three subclasses by hill station names. Subclasses extend the superclass & override methods famousfood() & famousfor(). Call the methods famousfood() & famousfor() by the Parent class i.e. Hillstations class. It should refer to the base class object & the base class method overrides the superclass method, base class method is invoked at runtime.

OBJECTIVE :-

1. To study method overloading
2. To study method overriding in Java.

THEORY :-

1. Method overloading & its advantages

It means defining multiple methods with the same name but different parameters (type, no, or order).

Advantages :-

- Improves code readability
- Increases flexibility

- Reduces code duplication
- Supports compile-time polymorphism

2. Method overriding & it's rule

It occurs when a subclass provides a specific implementation of a method already defined in it's superclass.

Rules :-

- Method name must be same
- Parameters must be the same
- Must be in inheritance relationship
- Cannot override final, static or private method
- Return type can be same or covariant

FAQS :-

1] Can we overload Java main() method?

⇒ Yes, but JVM will call only:
public static void main (String[] args)

2] Can we declare overloaded methods as final?

⇒ Yes, they can be declared as final.

3] Can overloaded method be overriden?

⇒ Yes, a method can be both overloaded & overridden

4] What is method overriding?

⇒ It is redefining a superclass method in a subclass with the same signature.

5] What happens if we change the arguments of overriding method?

⇒ It becomes method overloading, not overriding.

6] Can we change return type of overriding method?

⇒ Yes, but only if it is a covariant return type (subclass type).